Bohloko Family Farm Poultry Processing System - Class Diagrams

1. Introduction

This document presents comprehensive class diagrams for the Bohloko Family Farm Poultry Processing System using PlantUML notation. The diagrams follow UML 2.5 standards and illustrate the system architecture, class relationships, and design patterns implemented.

2. System Overview Diagram

```
@startuml SystemOverview

package "Bohloko Poultry Processing System" {
  package "User Management" {
    [User]
    [AuthenticationService]
    [UserRepository]
  }

  package "Production Management" {
    [ProductionCycle]
    [ProductionService]
    [ProductionRepository]
  }

  package "Inventory Management" {
    [InventoryItem]
    [InventoryService]
    [InventoryRepository]
  }

  package "Order Management" {
    [Order]
    [OrderService]
    [OrderRepository]
  }

  package "Analytics & Reporting" {
    [AnalyticsService]
    [ReportGenerator]
  }

  package "Compliance Management" {
    [ComplianceCheck]
    [ComplianceService]
  }
}
```

```
[User Management] --> [Production Management]
[Production Management] --> [Inventory Management]
[Inventory Management] --> [Order Management]
[Order Management] --> [Analytics & Reporting]
[Inventory Management] --> [Compliance Management]

@enduml
```

3. Core Domain Entities Diagram

```
@startuml CoreEntities

abstract class Entity {
  - id: String
  - createdAt: Date
  - updatedAt: Date
  + getId(): String
  + getCreatedAt(): Date
  + getUpdatedAt(): Date
  # updateTimestamp(): void
}

class Money {
  - amount: Number
  - currency: String
  + getAmount(): Number
  + getCurrency(): String
  + add(other: Money): Money
  + subtract(other: Money): Money
  + multiply(factor: Number): Money
  + equals(other: Money): Boolean
}

class Address {
  - street: String
  - city: String
  - state: String
  - country: String
  - postalCode: String
  + getFullAddress(): String
}

Entity <|-- User
Entity <|-- ProductionCycle
Entity <|-- InventoryItem
Entity <|-- Order
Entity <|-- Payment
Entity <|-- ComplianceCheck
Entity <|-- Certificate
```

```
@enduml
```

4. User Management Module Diagram

```
@startuml UserManagement

enum UserRole {
  FARM_MANAGER
  POULTRY_ATTENDANT
  PROCESSING_STAFF
  SALES_ASSISTANT
  CUSTOMER
}

enum UserType {
  CONSUMER
  RESTAURANT
  RETAILER
  DISTRIBUTOR
  FARM_GATE
  INSTITUTION
}

class UserProfile {
  - businessName: String
  - businessRegistrationNumber: String
  - taxId: String
  - address: Address
  - contact: ContactInfo
  - certifications: Certifications
}

abstract class User {
  - email: String
  - userType: UserType
  - profile: UserProfile
  - isActive: Boolean
  + getEmail(): String
  + getUserType(): UserType
  + getProfile(): UserProfile
  + isUserActive(): Boolean
  + activate(): void
  + deactivate(): void
  {abstract} getRole(): UserRole
}

class FarmStaffUser {
  - role: UserRole
  - department: String
```

```
  + getRole(): UserRole
  + getDepartment(): String
}

class CustomerUser {
  - loyaltyPoints: Number
  + getRole(): UserRole
  + getLoyaltyPoints(): Number
  + addLoyaltyPoints(points: Number): void
}

User <|-- FarmStaffUser
User <|-- CustomerUser

interface Authenticator {
  {abstract} authenticate(credentials: Any): Promise<String>
  {abstract} validateToken(token: String): Promise<Boolean>
}

interface PasswordHasher {
  {abstract} hash(password: String): Promise<String>
  {abstract} verify(password: String, hash: String): Promise<Boolean>
}

class JwtAuthenticator {
  + authenticate(credentials: Any): Promise<String>
  + validateToken(token: String): Promise<Boolean>
}

class BcryptPasswordHasher {
  + hash(password: String): Promise<String>
  + verify(password: String, hash: String): Promise<Boolean>
}

Authenticator <|.. JwtAuthenticator
PasswordHasher <|.. BcryptPasswordHasher

class UserService {
  - userRepository: UserRepository
  - authenticator: Authenticator
  - passwordHasher: PasswordHasher
  + registerUser(userData: Any): Promise<User>
  + authenticateUser(email: String, password: String): Promise<String>
}

interface UserRepository {
  {abstract} findById(id: String): Promise<User>
  {abstract} findByEmail(email: String): Promise<User>
  {abstract} save(user: User): Promise<void>
  {abstract} delete(id: String): Promise<void>
}

UserService --> UserRepository
```

```
UserService --> Authenticator
UserService --> PasswordHasher

@enduml
```

5. Production Management Module Diagram

```
@startuml ProductionManagement

enum ProductionType {
  BROILER_CYCLE
  EGG_PRODUCTION
  HATCHING
}

enum ProductionStatus {
  PLANNED
  IN_PROGRESS
  COMPLETED
  CANCELLED
}

enum FeedType {
  STARTER
  GROWER
  FINISHER
  LAYER
}

class ProductionBatch {
  - batchNumber: String
  - birdCount: Number
  - startDate: Date
  - expectedHarvestDate: Date
  + getAgeInDays(): Number
  + isReadyForHarvest(): Boolean
}

class DailyProductionLog {
  - date: Date
  - birdCount: Number
  - feedConsumed: Number
  - waterConsumed: Number
  - mortalityCount: Number
  - temperature: TemperatureRange
  - recordedBy: String
  + calculateMortalityRate(totalBirds: Number): Number
}

class MedicationRecord {
```

```
  - medicationName: String
  - dosage: String
  - administrationDate: Date
  - administeredBy: String
  - notes: String
}

class ProductionCycle {
  - cycleNumber: String
  - type: ProductionType
  - expectedDuration: Number
  - farmManagerId: String
  - batches: ProductionBatch[]
  - status: ProductionStatus
  - dailyLogs: DailyProductionLog[]
  + addBatch(batch: ProductionBatch): void
  + startCycle(): void
  + completeCycle(): void
  + addDailyLog(log: DailyProductionLog): void
  + getTotalBirds(): Number
  + getStatus(): ProductionStatus
}

class ProductionPlan {
  - expectedBirds: Number
  - expectedDuration: Number
  - budget: Money
  - startDate: Date
  + calculateDailyCost(): Money
}

interface ProductionOperations {
  {abstract} planCycle(plan: ProductionPlan): Promise<ProductionCycle>
  {abstract} startCycle(cycleId: String): Promise<void>
  {abstract} recordDailyLog(cycleId: String, log: DailyProductionLog):
Promise<void>
  {abstract} administerMedication(cycleId: String, medication:
MedicationRecord): Promise<void>
}

class ProductionService {
  - productionRepository: ProductionRepository
  - notificationService: NotificationService
  + planCycle(plan: ProductionPlan): Promise<ProductionCycle>
  + startCycle(cycleId: String): Promise<void>
  + recordDailyLog(cycleId: String, log: DailyProductionLog): Promise<void>
  + administerMedication(cycleId: String, medication: MedicationRecord):
Promise<void>
}

interface ProductionRepository {
  {abstract} findById(id: String): Promise<ProductionCycle>
  {abstract} save(cycle: ProductionCycle): Promise<void>
```

```
  {abstract} addMedication(cycleId: String, medication: MedicationRecord):
Promise<void>
}

ProductionOperations <|.. ProductionService
ProductionService --> ProductionRepository
ProductionService --> NotificationService

ProductionCycle "1" *-- "*" ProductionBatch
ProductionCycle "1" *-- "*" DailyProductionLog
ProductionCycle "1" *-- "*" MedicationRecord

@enduml
```

6. Inventory Management Module Diagram

```
@startuml InventoryManagement

enum ProductType {
  WHOLE_CHICKEN
  BREAST
  THIGHS
  WINGS
  DRUMSTICKS
  GIZZARDS
  FEET
}

class StorageLocation {
  - locationId: String
  - name: String
  - temperature: Number
  - capacity: Number
  - currentStock: Number
  + getAvailableCapacity(): Number
  + canStore(quantity: Number): Boolean
  + addStock(quantity: Number): void
  + removeStock(quantity: Number): void
}

class InventoryItem {
  - productType: ProductType
  - batchNumber: String
  - quantity: Number
  - unitWeight: Number
  - storageLocation: StorageLocation
  - harvestDate: Date
  - expiryDate: Date
  - unitCost: Money
  + getTotalWeight(): Number
```

```
  + getTotalCost(): Money
  + isExpired(): Boolean
  + daysUntilExpiry(): Number
  + split(quantity: Number): InventoryItem
}

class InventoryTransaction {
  - transactionId: String
  - itemId: String
  - transactionType: TransactionType
  - quantity: Number
  - reason: String
  - performedBy: String
  - timestamp: Date
}

interface InventoryOperations {
  {abstract} addItem(item: InventoryItem): Promise<void>
  {abstract} removeItem(itemId: String, quantity: Number, reason: String):
Promise<void>
  {abstract} transferItem(itemId: String, newLocation: StorageLocation,
quantity: Number): Promise<void>
  {abstract} getStockLevel(productType: ProductType): Promise<Number>
}

class InventoryService {
  - inventoryRepository: InventoryRepository
  - transactionLogger: TransactionLogger
  + addItem(item: InventoryItem): Promise<void>
  + removeItem(itemId: String, quantity: Number, reason: String): Promise<void>
  + transferItem(itemId: String, newLocation: StorageLocation, quantity:
Number): Promise<void>
  + getStockLevel(productType: ProductType): Promise<Number>
}

interface InventoryRepository {
  {abstract} findById(id: String): Promise<InventoryItem>
  {abstract} findByProductType(productType: ProductType):
Promise<InventoryItem[]>
  {abstract} save(item: InventoryItem): Promise<void>
  {abstract} delete(id: String): Promise<void>
}

interface TransactionLogger {
  {abstract} logTransaction(transaction: InventoryTransaction): Promise<void>
  {abstract} getTransactionHistory(itemId: String):
Promise<InventoryTransaction[]>
}

InventoryOperations <|.. InventoryService
InventoryService --> InventoryRepository
InventoryService --> TransactionLogger
```

```
InventoryItem "1" --> "1" StorageLocation
InventoryItem "1" --> "*" InventoryTransaction

@enduml
```

7. Order Management Module Diagram

```
@startuml OrderManagement

enum OrderStatus {
  PENDING
  CONFIRMED
  PROCESSING
  SHIPPED
  DELIVERED
  CANCELLED
}

enum PaymentMethod {
  CREDIT_CARD
  DEBIT_CARD
  BANK_TRANSFER
  MOBILE_MONEY
  CASH_ON_DELIVERY
}

enum ShippingMethod {
  STANDARD
  EXPRESS
  PICKUP
  FARM_GATE
  LOCAL_DELIVERY
}

class OrderItem {
  - productId: String
  - productName: String
  - quantity: Number
  - unitPrice: Money
  - batchNumber: String
  + getTotalPrice(): Money
}

class Order {
  - orderNumber: String
  - customerId: String
  - shippingAddress: Address
  - billingAddress: Address
  - paymentMethod: PaymentMethod
  - shippingMethod: ShippingMethod
```

```
  - items: OrderItem[]
  - status: OrderStatus
  + addItem(item: OrderItem): void
  + removeItem(productId: String): void
  + confirm(): void
  + cancel(reason: String): void
  + getTotalAmount(): Money
  + getItemCount(): Number
  + getStatus(): OrderStatus
}

class Payment {
  - orderId: String
  - amount: Money
  - method: PaymentMethod
  - status: PaymentStatus
  + process(): void
  + refund(reason: String): void
  + getStatus(): String
}

interface OrderOperations {
  {abstract} createOrder(orderData: Any): Promise<Order>
  {abstract} confirmOrder(orderId: String): Promise<void>
  {abstract} cancelOrder(orderId: String, reason: String): Promise<void>
  {abstract} processPayment(orderId: String, paymentData: Any):
Promise<Payment>
}

class OrderService {
  - orderRepository: OrderRepository
  - inventoryService: InventoryService
  - paymentProcessor: PaymentProcessor
  - notificationService: NotificationService
  + createOrder(orderData: Any): Promise<Order>
  + confirmOrder(orderId: String): Promise<void>
  + cancelOrder(orderId: String, reason: String): Promise<void>
  + processPayment(orderId: String, paymentData: Any): Promise<Payment>
}

interface OrderRepository {
  {abstract} findById(id: String): Promise<Order>
  {abstract} save(order: Order): Promise<void>
  {abstract} savePayment(payment: Payment): Promise<void>
}

interface PaymentProcessor {
  {abstract} process(payment: Payment, paymentData: Any): Promise<void>
  {abstract} refund(payment: Payment, reason: String): Promise<void>
}

OrderOperations <|.. OrderService
OrderService --> OrderRepository
```

```
OrderService --> InventoryService
OrderService --> PaymentProcessor
OrderService --> NotificationService

Order "1" *-- "*" OrderItem
Order "1" --> "1" Payment

@enduml
```

8. Analytics & Reporting Module Diagram

```
@startuml AnalyticsModule

interface ProductionAnalytics {
  {abstract} calculateMortalityRate(cycleId: String): Promise<Number>
  {abstract} calculateFeedConversionRatio(cycleId: String): Promise<Number>
  {abstract} calculateProductionCost(cycleId: String): Promise<Money>
}

interface SalesAnalytics {
  {abstract} calculateRevenue(startDate: Date, endDate: Date): Promise<Money>
  {abstract} calculateAverageOrderValue(): Promise<Money>
  {abstract} getTopProducts(limit: Number): Promise<ProductRevenue[]>
}

interface InventoryAnalytics {
  {abstract} calculateTurnoverRate(productType: ProductType): Promise<Number>
  {abstract} identifySlowMovingItems(thresholdDays: Number):
Promise<InventoryItem[]>
  {abstract} calculateWastePercentage(): Promise<Number>
}

class AnalyticsService {
  - productionRepository: ProductionRepository
  - orderRepository: OrderRepository
  - inventoryRepository: InventoryRepository
  + calculateMortalityRate(cycleId: String): Promise<Number>
  + calculateFeedConversionRatio(cycleId: String): Promise<Number>
  + calculateProductionCost(cycleId: String): Promise<Money>
  + calculateRevenue(startDate: Date, endDate: Date): Promise<Money>
  + calculateAverageOrderValue(): Promise<Money>
  + getTopProducts(limit: Number): Promise<ProductRevenue[]>
  + calculateTurnoverRate(productType: ProductType): Promise<Number>
  + identifySlowMovingItems(thresholdDays: Number): Promise<InventoryItem[]>
  + calculateWastePercentage(): Promise<Number>
}

ProductionAnalytics <|.. AnalyticsService
SalesAnalytics <|.. AnalyticsService
InventoryAnalytics <|.. AnalyticsService
```

```
AnalyticsService --> ProductionRepository
AnalyticsService --> OrderRepository
AnalyticsService --> InventoryRepository

class ReportGenerator {
  - analyticsService: AnalyticsService
  + generateProductionReport(cycleId: String): Promise<Report>
  + generateSalesReport(startDate: Date, endDate: Date): Promise<Report>
  + generateInventoryReport(): Promise<Report>
  + generateFinancialReport(period: DateRange): Promise<Report>
}

ReportGenerator --> AnalyticsService

@enduml
```

9. Compliance Management Module Diagram

```
@startuml ComplianceManagement

enum ComplianceCheckType {
  FOOD_SAFETY
  QUALITY_CONTROL
  SANITATION
  TEMPERATURE
}

class ComplianceCheck {
  - checkType: ComplianceCheckType
  - batchId: String
  - performedBy: String
  - result: CheckResult
  - notes: String
  - correctiveActions: String[]
  + getCheckType(): ComplianceCheckType
  + getResult(): String
  + requiresCorrectiveAction(): Boolean
  + addCorrectiveAction(action: String): void
}

class Certificate {
  - certificateNumber: String
  - batchId: String
  - issueDate: Date
  - expiryDate: Date
  - issuingAuthority: String
  - complianceChecks: ComplianceCheck[]
  + isValid(): Boolean
  + daysUntilExpiry(): Number
```

```
}

interface ComplianceOperations {
  {abstract} performCheck(checkData: Any): Promise<ComplianceCheck>
  {abstract} generateCertificate(batchId: String): Promise<Certificate>
  {abstract} generateComplianceReport(startDate: Date, endDate: Date):
Promise<Any>
}

class ComplianceService {
  - complianceRepository: ComplianceRepository
  - notificationService: NotificationService
  + performCheck(checkData: Any): Promise<ComplianceCheck>
  + generateCertificate(batchId: String): Promise<Certificate>
  + generateComplianceReport(startDate: Date, endDate: Date): Promise<Any>
}

interface ComplianceRepository {
  {abstract} saveCheck(check: ComplianceCheck): Promise<void>
  {abstract} saveCertificate(certificate: Certificate): Promise<void>
  {abstract} getChecksForBatch(batchId: String): Promise<ComplianceCheck[]>
  {abstract} getChecksInPeriod(startDate: Date, endDate: Date):
Promise<ComplianceCheck[]>
}

ComplianceOperations <|.. ComplianceService
ComplianceService --> ComplianceRepository
ComplianceService --> NotificationService

Certificate "1" *-- "*" ComplianceCheck

@enduml
```

10. Service Dependencies Diagram

```
@startuml ServiceDependencies

class UserService {
  + registerUser()
  + authenticateUser()
}

class ProductionService {
  + planCycle()
  + startCycle()
  + recordDailyLog()
}

class InventoryService {
  + addItem()
```

```
  + removeItem()
  + transferItem()
  + getStockLevel()
}

class OrderService {
  + createOrder()
  + confirmOrder()
  + cancelOrder()
  + processPayment()
}

class AnalyticsService {
  + calculateMortalityRate()
  + calculateRevenue()
  + calculateTurnoverRate()
}

class ComplianceService {
  + performCheck()
  + generateCertificate()
  + generateComplianceReport()
}

class NotificationService {
  + sendAlert()
  + notifyFarmManager()
  + sendOrderConfirmation()
}

UserService --> NotificationService
ProductionService --> NotificationService
OrderService --> NotificationService
ComplianceService --> NotificationService

OrderService --> InventoryService
AnalyticsService --> ProductionService
AnalyticsService --> OrderService
AnalyticsService --> InventoryService

@enduml
```

11. Repository Pattern Diagram

```
@startuml RepositoryPattern

interface Repository<T> {
  {abstract} findById(id: String): Promise<T>
  {abstract} save(entity: T): Promise<void>
  {abstract} delete(id: String): Promise<void>
```

```
}

Repository <|-- UserRepository
Repository <|-- ProductionRepository
Repository <|-- InventoryRepository
Repository <|-- OrderRepository
Repository <|-- ComplianceRepository

class UserRepository {
  + findByEmail(email: String): Promise<User>
}

class ProductionRepository {
  + addMedication(cycleId: String, medication: MedicationRecord): Promise<void>
}

class InventoryRepository {
  + findByProductType(productType: ProductType): Promise<InventoryItem[]>
}

class OrderRepository {
  + savePayment(payment: Payment): Promise<void>
}

class ComplianceRepository {
  + getChecksForBatch(batchId: String): Promise<ComplianceCheck[]>
  + getChecksInPeriod(startDate: Date, endDate: Date):
Promise<ComplianceCheck[]>
}

@enduml
```

12. Design Patterns Applied

12.1 Repository Pattern

- **Purpose**: Abstracts data access layer
- **Implementation**: Generic `Repository<T>` interface with concrete implementations
- **Benefits**: Decouples business logic from data storage, enables testing with mocks

12.2 Service Layer Pattern

- **Purpose**: Encapsulates business logic
- **Implementation**: `UserService`, `ProductionService`, `InventoryService`, etc.
- **Benefits**: Centralizes business rules, promotes reusability

12.3 Dependency Injection

- **Purpose**: Manages object dependencies
- **Implementation**: Constructor injection in all services

- **Benefits**: Improves testability, enables loose coupling

12.4 Strategy Pattern

- **Purpose**: Defines interchangeable algorithms
- **Implementation**: `Authenticator`, `PasswordHasher` interfaces
- **Benefits**: Enables swapping implementations (JWT vs OAuth, bcrypt vs argon2)

12.5 Factory Pattern

- **Purpose**: Creates objects without specifying exact class
- **Implementation**: `User` factory methods for different user types
- **Benefits**: Encapsulates object creation logic

12.6 Observer Pattern

- **Purpose**: Notifies multiple objects of state changes
- **Implementation**: `NotificationService` with multiple notification channels
- **Benefits**: Decouples subject from observers

13. Harvard Referencing

13.1 UML Standards and Notation

- OMG (Object Management Group). (2017). *Unified Modeling Language (UML) Version 2.5.1*. Available at: https://www.omg.org/spec/UML/2.5.1
- Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley.

13.2 Software Design Patterns

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Martin, R. C. (2002). *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall.

13.3 SOLID Principles

- Martin, R. C. (2000). *Design Principles and Design Patterns*. Object Mentor.
- Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

13.4 Domain-Driven Design

- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
- Vernon, V. (2013). *Implementing Domain-Driven Design*. Addison-Wesley.

13.5 Repository and Service Patterns

- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

- Nilsson, J. (2006). *Applying Domain-Driven Design and Patterns: With Examples in C# and .NET*. Addison-Wesley.

13.6 Agricultural Information Systems

- Sorensen, C. G., Fountas, S., Nash, E., Pesonen, L., Bochtis, D., Pedersen, S. M., ... & Blackmore, S. B. (2010). *Conceptual model of a future farm management information system*. Computers and Electronics in Agriculture, 72(1), 37-47.
- Kaloxylos, A., Eigenmann, R., Teye, F., Politopoulou, Z., Wolfert, S., Shrank, C., ... & Kormentzas, G. (2012). *Farm management systems and the Future Internet era*. Computers and Electronics in Agriculture, 89, 130-144.

14. Conclusion

The class diagrams presented in this document provide a comprehensive visual representation of the Bohloko Family Farm Poultry Processing System architecture. The design incorporates:

1. **Modular Architecture**: Clear separation into six functional modules
2. **SOLID Principles**: Adherence to software engineering best practices
3. **Design Patterns**: Implementation of proven patterns for maintainability
4. **Domain-Driven Design**: Focus on business domain modeling
5. **Scalability**: Architecture supporting future growth and extensions

The PlantUML diagrams serve as both documentation and implementation guidance, ensuring consistency between design and code. The Harvard referencing provides academic credibility and demonstrates research-based design decisions.

This architectural design provides a solid foundation for implementing a robust, maintainable, and scalable poultry processing system that meets the operational needs of Bohloko Family Farm while adhering to software engineering excellence standards.